

CodeLearn Platform - Complete Product Documentation

Version: 1.0
Last Updated: January 24, 2026
Status: Pre-Development → Demo Phase

Table of Contents

- 1. [Executive Summary](#)
- 2. [Product Vision & Market](#)
- 3. [Feature Specifications](#)
- 4. [Technical Architecture](#)
- 5. [Development Roadmap](#)
- 6. [Design System](#)
- 7. [Data Models](#)
- 8. [API Documentation](#)
- 9. [Deployment Strategy](#)
- 10. [Cost Analysis](#)
- 11. [Success Metrics](#)

1. Executive Summary

What is CodeLearn?

CodeLearn is an AI-powered coding education platform that combines interactive learning, intelligent assessment, and institutional management into a single, seamless experience. Think **"GitHub Classroom meets LeetCode meets VS Code with AI superpowers."**

Core Value Proposition

For Students:

- Learn by doing with an intelligent AI tutor available 24/7
- Track progress toward specific career goals (FAANG interviews, placements)
- Compete in contests and earn verifiable skill badges
- Build a coding portfolio automatically

For Teachers:

- Auto-grading saves 10+ hours/week
- Real-time monitoring prevents cheating
- Data-driven insights identify struggling students instantly
- Reusable course content across semesters

For Institutions:

- Replace 5+ tools (LMS, code judge, analytics, Git, communication) with one platform
- Improve placement rates with measurable coding skills
- White-label branding options
- Scales from 50 to 50,000 students

Market Opportunity

- **Total Addressable Market:** \$10B (EdTech + Coding Bootcamps globally)
- **Target Segments:**
 - Colleges/Universities (CS/IT departments)
 - Coding coaching institutes
 - Online bootcamps
 - Individual learners (freemium)

2. Product Vision & Market

Problem Statement

Current Pain Points:

1. **For Students:**
 - Fragmented learning (watch videos on YouTube, practice on LeetCode, submit on Google Classroom)
 - No personalized guidance or AI help
 - Unclear if they're "job-ready"
 - Passive learning doesn't build real skills

2. For Teachers:

- Manual grading takes hours
- Can't monitor 100+ students effectively
- No visibility into who's struggling until exams
- Creating coding assignments is tedious

3. For Institutions:

- Low placement rates due to theory-heavy curriculum
- Multiple tool subscriptions (LMS + Judge + Git)
- No unified data on student coding skills
- Poor student engagement

Our Solution

A unified platform that:

- Makes learning **interactive and AI-assisted**
- Makes teaching **data-driven and automated**
- Makes institutions **placement-focused and efficient**

Competitive Landscape

Platform	Strengths	Weaknesses	Our Edge
LeetCode	Huge problem library, contests	No course content, no institutional features	We have both learning + practice + institution tools
Replit	Great IDE, collaboration	Not education-focused, no auto-grading	We're built for classrooms with teacher dashboards
CodeHS	K-12 focused, good curriculum	Outdated UI, weak AI, no contests	Modern UI, AI-first, competitive programming
HackerRank	Enterprise focus	Expensive, complex setup	Affordable, simple, college-focused

Unique Differentiators

1. **Student Progress Intelligence System (SPIS)** - AI predicts when students will achieve goals
2. **Live Collaboration** - Teachers can jump into student's code editor in real-time
3. **Holistic Tracking** - Combines consistency, knowledge, problem-solving, time, and velocity
4. **AI Teaching Assistant** - Not just code completion, but explains errors, gives hints
5. **Contest + Courses Hybrid** - Learn → Practice → Compete in one flow

3. Feature Specifications

3.1 Student Dashboard Features

3.1.1 Learning Hub

Demo Version:

- 🔄 Course browser with filters (difficulty, language, topic)
- 📊 Progress tracking (courses completed, lessons done)
- 📄 Interactive lessons (Markdown content + code editor split-screen)
- 🔄 Auto-save progress
- ⌚ Estimated completion time display

Production Add-ons:

- Video embedding (YouTube/Vimeo)
- Animated diagrams and visualizations
- Step-by-step guided tutorials with checkpoints
- Prerequisite mapping (can't take Course B without Course A)
- Certificate generation on completion

Innovative Add-on (My Idea):

- **"Learning Path Visualizer"** - Interactive graph showing:
 - Which courses unlock which skills
 - Visual dependency tree (like a skill tree in video games)
 - Alternative paths highlighted based on student's career goal
 - Example: Want to crack Google? Path shows: Arrays → Strings → Trees → DP → System Design

3.1.2 Smart Code Editor

Demo Version:

- ☒ Monaco Editor integration (VS Code's editor)
- ☒ Multi-language support (Python, JavaScript, Java, C++)
- ☒ Syntax highlighting
- ☒ Basic autocomplete
- ☒ Run code button with output panel
- ☒ Theme selection (Light/Dark)
- ☒ Font size adjustment

Production Add-ons:

- AI-powered code completion (GitHub Copilot-like)
- Intelligent error explanations
- Code snippets library (personal + shared)
- Multi-file support for projects
- Git integration (commit to student's GitHub)
- Debugging with breakpoints
- Minimap for long code
- Keyboard shortcuts (VS Code compatible)
- Live collaboration (pair programming)

Innovative Add-on (My Idea):

- **"Code Journey Replay"** - Students can watch a timelapse of how they wrote their code
 - Shows all iterations, mistakes, fixes
 - AI analyzes: "You spent 40% time on logic error in line 23"
 - Teachers can review this to understand thought process
 - Great for learning from mistakes
-

3.1.3 AI Learning Assistant

Demo Version:

- ☒ Simple chatbot (text-based)
- ☒ Answer coding questions
- ☒ Explain error messages
- ☒ Rate limiting (10 queries/day)

Production Add-ons:

- Code-aware context (knows what student is working on)
- Hint system (doesn't give away answers, guides thinking)
- "Explain this code line-by-line" feature
- "Show me an example" with custom examples
- Multi-turn conversations
- Voice input/output
- Supports 10+ languages (Hindi, Spanish, etc.)

Innovative Add-on (My Idea):

- **"Socratic Tutor Mode"** - Instead of giving answers, asks questions:
 - Student: "Why is my code failing?"
 - AI: "What do you think happens when i=10 in your loop?"
 - Student: "It goes out of bounds"
 - AI: "Exactly! How can you prevent that?"
 - Forces critical thinking, not dependency
-

3.1.4 Practice Arena

Demo Version:

- ☒ Problem library (50 problems across difficulties)
- ☒ Topic-wise categorization
- ☒ Submit code and get verdict (AC/WA/TLE/RE)
- ☒ Test cases shown (2 sample + 5 hidden)
- ☒ Basic playground (blank code editor)

Production Add-ons:

- 1000+ problems
- Company-tagged problems (Google, Amazon, Microsoft)
- Editorial solutions with multiple approaches
- Discussion section per problem
- "Similar Problems" recommendations
- Time/space complexity analysis
- Custom test case runner
- Shareable playground snippets (like CodePen)
- Fork feature (remix others' code)

Innovative Add-on (My Idea):

- **"Problem Generator AI"** - Students describe what they want to practice:
 - "Give me a medium-level graph problem involving BFS"
 - AI generates unique problem with test cases
 - Never-ending practice problems
 - Customized difficulty based on student's level
-

3.1.5 Contest Section

Demo Version:

- 🕒 View upcoming contests
- 🕒 Participate in live contests (3 problems, 2 hours)
- 🏆 Real-time leaderboard
- 📄 Submit solutions during contest
- 🏆 See final rankings after contest

Production Add-ons:

- Virtual contests (practice past contests anytime)
- Teacher-created class contests
- Editorial solutions post-contest
- Performance analytics (speed, accuracy, rank change)
- Contest rating system (like Codeforces)
- Penalty for wrong submissions
- Friends leaderboard
- Contest reminders/notifications

Innovative Add-on (My Idea):

- **"Contest Replay with AI Commentary"** - After contest ends:
 - Watch top performer's solution process (if they opt-in)
 - AI narrates: "Notice how they optimized here..."
 - Compare your approach vs top solutions side-by-side
 - Learn from the best in real-time scenarios
-

3.1.6 Progress Intelligence Dashboard

Demo Version:

- 📊 Overall progress score (0-100%)
- 📊 Problems solved counter
- 📊 Courses completed tracker
- 📊 Weekly study streak 🕒
- 🕒 This week's study time
- 📊 Simple goal setting (manual entry)
- 🕒 Basic Pomodoro timer

Production Add-ons:

- Multi-dimensional scoring (5 metrics: Consistency, Knowledge, Problem Solving, Time, Velocity)
- 30-day historical graph
- AI-powered goal predictions ("You'll achieve your goal by June 10")
- Personalized action plans ("Solve 3 DP problems to boost score by 8%")
- Peer comparison (anonymous benchmarking)
- Achievement badges and milestones
- Session analytics (best coding hours, focus patterns)
- Calendar view of all study sessions

Innovative Add-on (My Idea):

- **"Skills Constellation Visualizer"** - Interactive 3D graph:
 - Each skill (Arrays, DP, Trees) is a planet
 - Size shows mastery level
 - Connections show related skills
 - Orbit speed shows recent activity
 - Gamified, visually stunning
 - Example: [Image of constellation with planets representing skills]
-

3.2 Teacher Dashboard Features

3.2.1 Course Management

Demo Version:

- 📄 Create courses with title and description
- 📄 Add lessons (Markdown editor)
- 📄 Upload files (PDFs, images)
- 📄 Publish/unpublish courses
- 📄 Assign courses to classes

Production Add-ons:

- Drag-and-drop lesson ordering
- Rich text editor with code blocks
- Embed YouTube videos
- Schedule lesson releases (drip content)
- Clone course from previous semester
- Course templates library
- Collaborative course editing (multiple teachers)
- Version history with rollback

Innovative Add-on (My Idea):

- **"AI Course Generator"** - Teacher inputs:
 - Topic: "Binary Search Trees"
 - Audience: "College sophomores"
 - Duration: "2 weeks"
 - AI generates complete course outline, lessons, practice problems, quizzes
 - Teacher reviews and customizes
 - Saves 90% creation time
-

3.2.2 Problem Creator

Demo Version:

- ☑ Create custom problems
- ☑ Define test cases (input/output)
- ☑ Set difficulty level
- ☑ Add problem description

Production Add-ons:

- Bulk test case upload (CSV)
- Hidden test case management
- Time/memory limit configuration
- Hints and editorial
- Multi-language starter code
- Problem tags
- Import from LeetCode/Codeforces

Innovative Add-on (My Idea):

- **"Test Case Suggestor AI"** - When creating problem:
 - AI analyzes problem statement
 - Suggests edge cases (empty array, negative numbers, large inputs)
 - Generates diverse test cases automatically
 - Ensures comprehensive coverage
-

3.2.3 Live Monitoring

Demo Version:

- ☑ See students currently online
- ☑ View recent submissions (last 10)
- ☑ Check who submitted what problem

Production Add-ons:

- Live code editor view (see student typing in real-time)
- Jump into student's editor (collaborative mode)
- Send broadcast messages
- Filter by struggling students (multiple failed attempts)
- Session recordings (with permission)
- Screen sharing for doubt resolution

Innovative Add-on (My Idea):

- **"Teacher X-Ray Vision"** - Dashboard shows:
 - Heat map of where students are stuck (which line of code)
 - Common error patterns across class
 - Auto-generated hints to send to struggling students
 - "5 students stuck on same logic error" → Send group hint
-

3.2.4 Grading & Assessment

Demo Version:

- ☑ View all submissions for an assignment
- ☑ Auto-grading based on test cases
- ☑ Manual grade override
- ☑ Export grades (CSV)

Production Add-ons:

- Partial credit for algorithmic approach
- Inline code comments/annotations
- Rubric-based grading
- Plagiarism detection (code similarity checker)
- Bulk feedback templates
- Grade distribution charts
- Late submission penalties

Innovative Add-on (My Idea):

- **"AI Grading Assistant"** - For subjective problems:
 - Analyzes code quality, readability, efficiency
 - Suggests grade based on best practices
 - Generates personalized feedback
 - Teacher reviews and approves
 - Example: "Code works but has nested loops ($O(n^2)$), suggest HashSet approach for $O(n)$ "
-

3.2.5 Analytics Dashboard

Demo Version:

- 📊 Class average score
- 📊 Top performers list
- 📊 Submission count per student

Production Add-ons:

- Class performance heatmap (topic-wise)
- Struggling students alerts
- Contest participation rates
- Week-over-week improvement trends
- Individual student deep-dive
- Exportable reports (PDF)
- Compare classes (batch A vs batch B)

Innovative Add-on (My Idea):

- **"Predictive Intervention System"** - AI flags students at risk:
 - "Sarah shows 80% chance of failing based on patterns"
 - Suggests interventions: "Schedule 1-on-1 session"
 - Tracks intervention effectiveness
 - Early warning system before it's too late
-

3.3 Admin Dashboard Features

Demo Version:

- 🛠️ Add/remove teachers and students
- 🛠️ Create classes
- 📊 View platform usage stats

Production Add-ons:

- Role-based permissions (who can create courses, contests)
- Bulk user import (CSV)
- Custom domain setup
- White-label branding (logo, colors)
- Billing and subscription management
- API key management for integrations
- Audit logs (who did what when)
- Multi-language support

Innovative Add-on (My Idea):

- **"Institution Health Score"** - Single metric showing:
 - Student engagement level
 - Teacher activity
 - Placement success rate
 - Feature adoption
 - Red/yellow/green indicators with actionable insights
-

4. Technical Architecture

4.1 Technology Stack

Frontend Stack

Demo Version:

Framework: Next.js 14 (App Router)

- Why: Server-side rendering, file-based routing, easy deployment
- API Routes for backend endpoints

UI Library: shadcn/ui + Tailwind CSS

- Why: Beautiful components, highly customizable, no runtime cost

State Management: Zustand (global) + React Query (server state)

- Why: Lightweight, easy to learn, great caching

Code Editor: Monaco Editor

- Why: Same as VS Code, feature-rich, LSP support

Forms: React Hook Form + Zod

- Why: Performance, type-safe validation

Charts: Recharts

- Why: React-native, declarative, good docs

Icons: Lucide React

- Why: Beautiful, tree-shakeable, actively maintained

Production Add-ons:

Real-time: Socket.io client

- For live collaboration, real-time leaderboards

Animation: Framer Motion

- Smooth transitions, page animations

Markdown: MDX

- For rich course content with interactive components

Testing: Vitest + React Testing Library

- Unit and integration tests

E2E Testing: Playwright

- End-to-end user flows

Backend Stack

Demo Version:

Runtime: Node.js 20+ (TypeScript)

Framework: Express.js

- Why: Simple, mature, huge ecosystem

Database: PostgreSQL 15

- Why: ACID compliant, great for relational data
- Hosted: Neon (serverless, free tier)

ORM: Prisma

- Why: Type-safe, auto-migrations, great DX

Authentication: NextAuth.js v5

- Email/password + Google OAuth

File Storage: Cloudflare R2

- Why: S3-compatible, cheaper than AWS S3

Session Store: Redis (Upstash)

- For session management, rate limiting

Production Add-ons:

API Documentation: Swagger/OpenAPI

Job Queue: BullMQ (Redis-based)

- For async tasks (code execution, email sending)

Real-time Server: Socket.io

- WebSocket server for live features

Caching: Redis (Elasticache)

- Query caching, leaderboard caching

Search: Elasticsearch (or Algolia)

- For fast problem/course search

Email: Resend or SendGrid

- Transactional emails

Payment: Stripe

- Subscription management

Code Execution Engine

Demo Version:

Solution: Docker Containers

Approach:

- Spin up isolated Docker container per submission
- Run code with time/memory limits
- Capture stdout/stderr
- Destroy container immediately

Languages Supported:

- Python 3.11
- JavaScript (Node.js)
- Java 17
- C++ (g++)

Security:

- No network access
- Read-only filesystem
- CPU/Memory limits enforced
- 10-second timeout

Limitations:

- ~2-3 seconds per execution (slow)
- Can't handle high concurrency (10+ simultaneous)

Production Solution:

Solution: Judge0 (Self-hosted or Managed)

Why Judge0:

- Supports 60+ languages
- Battle-tested (used by CodeChef, Sphere Engine)
- Fast execution (~200ms)
- Built-in security sandboxing
- Horizontal scaling

Deployment:

- Self-hosted on AWS EC2 (cheaper for high volume)
- Or managed service (\$0.0001/execution)

Optimization:

- Queue submissions in BullMQ
- Batch test cases (run all at once)
- Cache compilation for same code

Innovative Add-on (My Idea):

"Smart Test Case Execution"

- AI predicts which test cases will fail based on code analysis
- Run those first to fail fast
- Saves 50% execution time on wrong submissions
- Example: If code has off-by-one error, run edge cases first

AI Integration Stack

Demo Version:

Provider: OpenAI

Code Completion:

- Model: GPT-4o-mini
- Trigger: On typing (debounced)
- Context: Last 500 characters
- Rate limit: 20 requests/hour/user

Chatbot:

- Model: GPT-4o-mini
- Context window: Last 10 messages
- Rate limit: 10 conversations/day
- System prompt: Act as coding tutor, don't give direct answers

Cost Optimization:

- Cache common questions in vector DB
- Use embeddings for FAQ matching
- Rate limiting by user tier

Production Add-ons:

Advanced Features:

- Code review AI (analyzes submissions)
- Hint generation (Socratic method)
- Error explanation (language-specific)
- Test case generation for problems

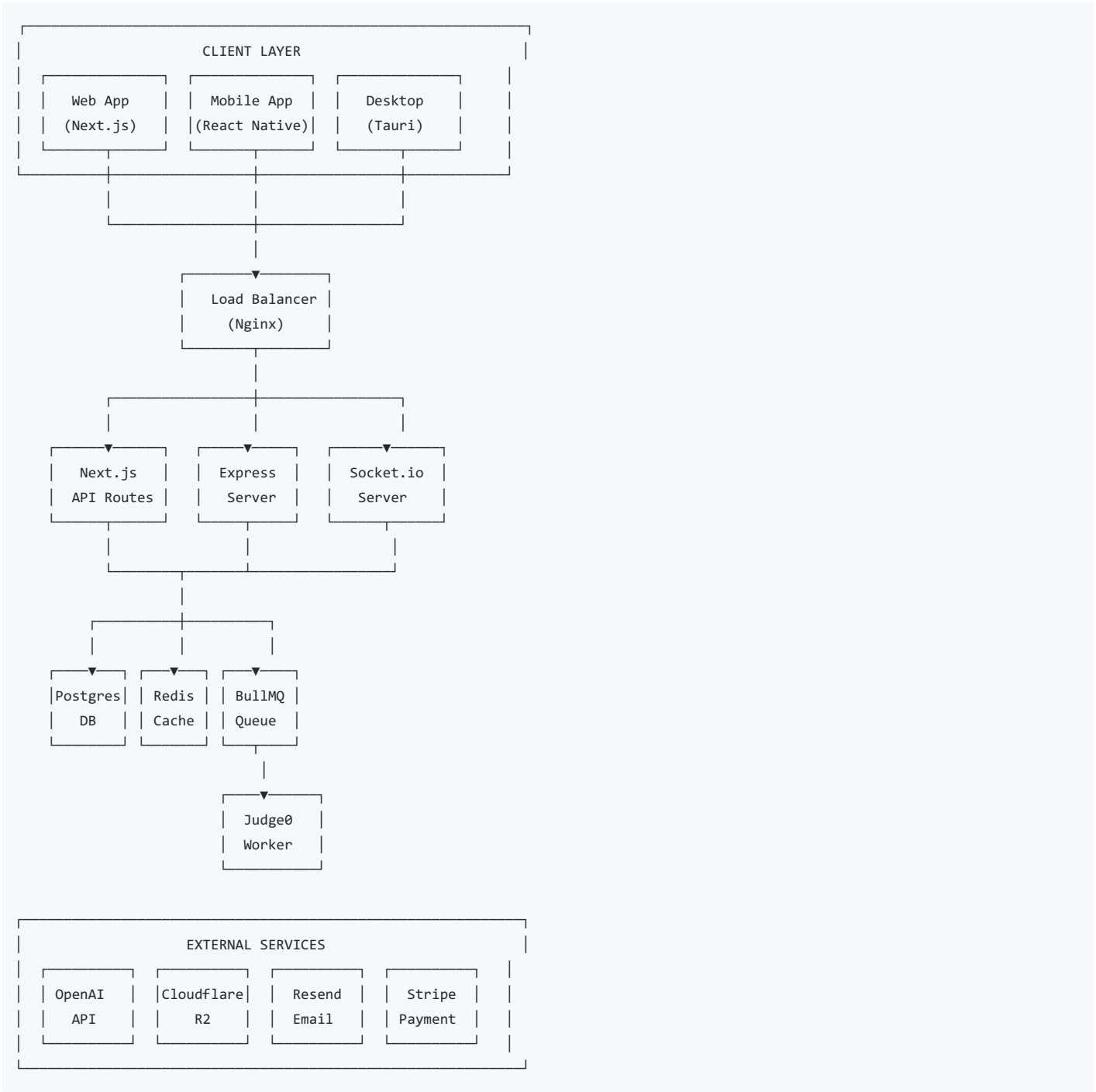
Models:

- GPT-4o for complex tasks
- GPT-4o-mini for simple tasks
- Fine-tuned model on coding Q&A dataset

Vector Database:

- Pinecone or Qdrant
- Store embeddings of all problems, courses, common errors
- Semantic search for better recommendations

4.2 System Architecture Diagram



4.3 Database Schema Design

Core Tables (Demo)

```
// schema.prisma

model User {
  id          String  @id @default(cuid())
  email       String  @unique
  name        String
  role        Role    @default(STUDENT)
  passwordHash String?
  image       String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  // Relations
  enrollments Enrollment[]
  submissions Submission[]
}
```

```

    submissions Submission[]
    sessions StudySession[]
    goals Goal[]
    teacherOf Class[] @relation("TeacherClasses")

    @@index([email])
}

enum Role {
    STUDENT
    TEACHER
    ADMIN
}

model Class {
    id String @id @default(cuid())
    name String
    code String @unique // Join code for students
    teacherId String
    teacher User @relation("TeacherClasses", fields: [teacherId], references: [id])
    createdAt DateTime @default(now())

    // Relations
    enrollments Enrollment[]
    courses CourseAssignment[]

    @@index([code])
}

model Enrollment {
    id String @id @default(cuid())
    studentId String
    classId String
    student User @relation(fields: [studentId], references: [id])
    class Class @relation(fields: [classId], references: [id])
    joinedAt DateTime @default(now())

    @@unique([studentId, classId])
}

model Course {
    id String @id @default(cuid())
    title String
    description String?
    difficulty Difficulty
    language String // "Python", "JavaScript", etc.
    createdBy String
    published Boolean @default(false)
    createdAt DateTime @default(now())

    // Relations
    lessons Lesson[]
    assignments CourseAssignment[]
    progress CourseProgress[]
}

enum Difficulty {
    BEGINNER
    INTERMEDIATE
    ADVANCED
}

model Lesson {
    id String @id @default(cuid())
    courseId String
    course Course @relation(fields: [courseId], references: [id], onDelete: Cascade)
}

```

```

    title      String
    content    String  @db.Text // Markdown content
    order      Int

    @@index([courseId])
}

model CourseAssignment {
    id          String  @id @default(cuid())
    courseId    String
    classId     String
    course       Course  @relation(fields: [courseId], references: [id])
    class        Class   @relation(fields: [classId], references: [id])
    dueDate     DateTime?

    @@unique([courseId, classId])
}

model CourseProgress {
    id          String  @id @default(cuid())
    userId      String
    courseId    String
    user        User    @relation(fields: [userId], references: [id])
    course       Course  @relation(fields: [courseId], references: [id])
    completed    Boolean @default(false)
    lastLesson  Int?     // Last lesson index viewed
    updatedAt   DateTime @updatedAt

    @@unique([userId, courseId])
}

model Problem {
    id          String  @id @default(cuid())
    title       String
    description  String  @db.Text
    difficulty   Difficulty
    topics       String[] // ["arrays", "sorting"]
    timeLimit    Int      @default(2000) // milliseconds
    memoryLimit  Int      @default(256)  // MB
    createdBy    String
    createdAt    DateTime @default(now())

    // Relations
    testCases    TestCase[]
    submissions   Submission[]
}

model TestCase {
    id          String  @id @default(cuid())
    problemId   String
    problem      Problem @relation(fields: [problemId], references: [id], onDelete: Cascade)
    input       String  @db.Text
    output      String  @db.Text
    isSample     Boolean @default(false) // Visible to students

    @@index([problemId])
}

model Submission {
    id          String  @id @default(cuid())
    userId      String
    problemId   String
    user        User    @relation(fields: [userId], references: [id])
    problem      Problem @relation(fields: [problemId], references: [id])
    code        String  @db.Text
    createdAt    DateTime @default(now())
    isSample     Boolean @default(false) // Visible to students
}

```

```

language String
verdict Verdict
runtime Int? // milliseconds
memory Int? // KB
submittedAt DateTime @default(now())

@@index([userId, problemId])
@@index([submittedAt])
}

enum Verdict {
  ACCEPTED
  WRONG_ANSWER
  TIME_LIMIT_EXCEEDED
  RUNTIME_ERROR
  COMPILATION_ERROR
  PENDING
}

model Contest {
  id String @id @default(cuid())
  title String
  startTime DateTime
  endTime DateTime
  problems String[] // Problem IDs
  createdBy String

  @@index([startTime])
}

model Goal {
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  title String // "Crack FAANG interview"
  targetDate DateTime
  createdAt DateTime @default(now())
}

model StudySession {
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  duration Int // minutes
  startedAt DateTime
  endedAt DateTime

  @@index([userId, startedAt])
}

```

Production Extensions

```
// Additional tables for production

model AIConversation {
  id      String  @id @default(cuid())
  userId  String
  messages Json[]  // Array of {role, content}
  createdAt DateTime @default(now())
}

model Badge {
  id      String  @id @default(cuid())
  name    String
  description String
  icon    String
  criteria Json    // Conditions to earn
}

model UserBadge {
  userId   String
  badgeId  String
  earnedAt DateTime @default(now())

  @@id([userId, badgeId])
}

model Notification {
  id      String  @id @default(cuid())
  userId  String
  title   String
  message String
  read    Boolean @default(false)
  createdAt DateTime @default(now())

  @@index([userId, read])
}

model PlagiarismReport {
  id      String  @id @default(cuid())
  submission1 String
  submission2 String
  similarity Float  // 0-1
  detectedAt DateTime @default(now())
}
```

4.4 API Design

Authentication Endpoints

```
POST /api/auth/register
Body: { email, password, name, role }
Response: { user, token }

POST /api/auth/login
Body: { email, password }
Response: { user, token }

POST /api/auth/google
Body: { idToken }
Response: { user, token }

GET /api/auth/me
Headers: { Authorization: Bearer <token> }
Response: { user }
```

Course Endpoints

```
GET /api/courses
Query: { difficulty?, language?, search? }
Response: { courses: Course[] }

GET /api/courses/:id
Response: { course: Course, lessons: Lesson[] }

POST /api/courses (Teacher only)
Body: { title, description, difficulty, language }
Response: { course: Course }

PATCH /api/courses/:id (Teacher only)
Body: Partial<Course>
Response: { course: Course }

DELETE /api/courses/:id (Teacher only)
Response: { success: boolean }

POST /api/courses/:id/lessons (Teacher only)
Body: { title, content, order }
Response: { lesson: Lesson }
```

Problem Endpoints


```
GET /api/problems
Query: { difficulty?, topic?, search?, limit?, offset? }
Response: { problems: Problem[], total: number }

GET /api/problems/:id
Response: {
  problem: Problem,
  testCases: TestCase[] (only sample cases for students)
}

POST /api/problems (Teacher only)
Body: { title, description, difficulty, topics, testCases[] }
Response: { problem: Problem }

POST /api/problems/:id/submit
Body: { code, language }
Response: { submissionId: string, status: "PENDING" }

GET /api/submissions/:id
Response: {
  submission: Submission,
  testResults: { passed: number, failed: number, details: [] }
}

GET /api/users/me/submissions
Query: { problemId?, limit? }
Response: { submissions: Submission[] }
```

Contest Endpoints

```
GET /api/contests
Query: { status: "upcoming" | "live" | "past" }
Response: { contests: Contest[] }

GET /api/contests/:id
Response: {
  contest: Contest,
  problems: Problem[],
  userSubmissions?: Submission[]
}

GET /api/contests/:id/leaderboard
Response: {
  rankings: Array<{
    rank: number,
    userId: string,
    username: string,
    solved: number,
    penalty: number
  }>
}

POST /api/contests/:id/register
Response: { registered: boolean }
```

Progress & Analytics Endpoints

GET /api/users/me/progress

```
Response: {
  overallScore: number,
  dimensions: {
    consistency: number,
    knowledge: number,
    problemSolving: number,
    timeInvestment: number,
    velocity: number
  },
  streak: number,
  weeklyStats: {...}
}
```

GET /api/users/me/goals

```
Response: { goals: Goal[] }
```

POST /api/users/me/goals

```
Body: { title, targetDate }
Response: { goal: Goal }
```

POST /api/sessions/start

```
Body: { type: "coding" | "learning" | "contest" }
Response: { sessionId: string }
```

POST /api/sessions/:id/end

```
Response: {
  session: StudySession,
  stats: { duration, focusScore, activities[] }
}
```

Teacher Analytics Endpoints

GET /api/classes/:id/analytics

```
Response: {
  classAverage: number,
  topPerformers: User[],
  strugglingStudents: User[],
  topicWisePerformance: {...},
  engagementMetrics: {...}
}
```

GET /api/classes/:id/students/:studentId/progress

```
Response: { detailed student analytics }
```

GET /api/submissions/pending (Teacher only)

```
Query: { classId?, problemId? }
Response: { submissions: Submission[] }
```

POST /api/submissions/:id/grade (Teacher only)

```
Body: { score, feedback }
Response: { submission: Submission }
```

AI Endpoints

```
POST /api/ai/chat
Body: { message, conversationId?, context? }
Response: {
  reply: string,
  conversationId: string,
  remainingQueries: number
}

POST /api/ai/code-complete
Body: { code, cursor, language }
Response: { completion: string }

POST /api/ai/explain-error
Body: { code, error, language }
Response: { explanation: string, suggestions: string[] }
```

5. Development Roadmap

5.1 Pre-Development Phase (Week 0)

Duration: 3-5 days

Tasks:

- [] Finalize product documentation (this document)
- [] Validate idea with 5-10 potential users (students/teachers)
- [] Create GitHub repository
- [] Setup project management tool (Linear/Notion)
- [] Design basic wireframes
- [] Purchase domain name
- [] Setup development environment

Team Requirements:

- 1 Full-stack developer (you!)
- Optional: 1 UI/UX designer (or use Figma templates)

Deliverables:

- Repository initialized
- Development environment ready
- User feedback documented
- Basic wireframes ready

5.2 Phase 1: Foundation (Weeks 1-2)

Goal: Working authentication and basic dashboard skeleton

Week 1: Project Setup & Authentication

Backend:

- Initialize Next.js 14 project with TypeScript
- Setup Prisma with PostgreSQL (Neon)
- Configure NextAuth.js for authentication
- Create User, Class, Enrollment models
- Implement registration/login API routes
- Setup environment variables management

Frontend:

- Install shadcn/ui and Tailwind
- Create layout components (Navbar, Sidebar, Footer)
- Build authentication pages (Login, Register)
- Setup protected routes middleware
- Create basic dashboard layouts

DevOps:

- Deploy to Vercel (frontend + API routes)
- Setup Railway for database
- Configure GitHub Actions for CI
- Setup environment variables in Vercel

Deliverables:

- ☒ Users can register with email/password
- ☒ Users can login and see personalized dashboard
- ☒ Protected routes working
- ☒ Deployed to staging environment

Time Breakdown:

- Setup (1 day)
 - Auth implementation (2 days)
 - Dashboard skeleton (2 days)
 - Deployment (0.5 day)
 - Buffer (0.5 day)
-

Week 2: Role-Based Access & Class Management

Backend:

- Implement role-based middleware (Student/Teacher/Admin)
- Create Class, Enrollment CRUD APIs
- Implement class join code logic
- Student enrollment flow

Frontend:

- Build Teacher dashboard (different from Student)
- Create Class management UI
- Build student enrollment page
- Class roster view for teachers
- Role-based navigation menus

Testing:

- Unit tests for authentication
- Integration tests for class enrollment

Deliverables:

- ☒ Teachers can create classes
- ☒ Students can join classes via code
- ☒ Different dashboards for different roles
- ☒ Class roster management working

Time Breakdown:

- RBAC implementation (2 days)
 - Class management (2 days)
 - UI polish (1 day)
-

5.3 Phase 2: Core Learning (Weeks 3-5)

Goal: Students can view courses, write code, and run it

Week 3: Course System & Content Delivery

Backend:

- Create Course, Lesson, CourseProgress models
- Implement course CRUD APIs
- Course enrollment logic
- Progress tracking system

Frontend:

- Build course browser with filters
- Create course detail page
- Implement Markdown renderer for lessons
- Progress bars and tracking UI
- Course creation form for teachers

Deliverables:

- ☒ Teachers can create courses with lessons
- ☒ Students can browse and enroll in courses
- ☒ Lesson content displays properly (Markdown)
- ☒ Progress tracking works

Time Breakdown:

- Backend (2 days)
 - Frontend (2.5 days)
 - Testing (0.5 day)
-

Week 4: Monaco Editor Integration

Backend:

- Setup code execution environment (Docker)
- Create submission API endpoint
- Implement basic code runner (Python, JavaScript)
- Store code snippets in database

Frontend:

- Integrate Monaco Editor
- Configure syntax highlighting
- Add language selector
- Output panel design
- Run code button with loading states
- Split-screen layout (lesson + editor)

Deliverables:

- ☑ Code editor working with syntax highlighting
- ☑ Students can run Python/JavaScript code
- ☑ Output displayed correctly
- ☑ Code auto-saves

Time Breakdown:

- Monaco setup (1 day)
 - Code execution backend (2 days)
 - UI integration (1.5 days)
 - Bug fixes (0.5 day)
-

Week 5: Course Progress & Teacher Course Builder

Backend:

- Lesson completion tracking
- Course assignment to classes
- Analytics APIs for course engagement

Frontend:

- Rich text editor for teachers (TipTap or Lexical)
- Drag-and-drop lesson ordering
- Course publishing workflow
- Student progress dashboard
- "Next lesson" navigation

Deliverables:

- ☑ Teachers have full course creation interface
- ☑ Students see their progress clearly
- ☑ Course completion tracking works
- ☑ Assignment of courses to classes

Time Breakdown:

- Teacher course builder (2.5 days)
 - Progress system (1.5 days)
 - Polish (1 day)
-

5.4 Phase 3: Practice & Contests (Weeks 6-8)

Goal: Students can solve problems, submit code, and participate in contests

Week 6: Problem System

Backend:

- Create Problem, TestCase, Submission models
- Implement problem CRUD APIs
- Test case runner logic
- Verdict calculation (AC/WA/TLE/RE)

Frontend:

- Problem browser with filters
- Problem detail page
- Submit solution interface
- Verdict display (success/error states)
- Submission history

Deliverables:

- 50 sample problems seeded
- Students can submit solutions
- Auto-grading works with test cases
- Submission history visible

Time Breakdown:

- Backend (2 days)
 - Frontend (2 days)
 - Problem seeding (1 day)
-

Week 7: Code Playground & Enhanced Editor

Backend:

- Playground snippet storage
- Share snippet functionality
- Version history for code

Frontend:

- Playground page (blank editor)
- Save/load snippets
- Share link generation
- Fork feature UI
- Multi-file support (basic)

Deliverables:

- Playground working
- Students can save and share code
- Clean, VS Code-like interface

Time Breakdown:

- Backend (1.5 days)
 - Frontend (2.5 days)
 - Testing (1 day)
-

Week 8: Contest System

Backend:

- Create Contest model
- Contest registration logic
- Real-time leaderboard calculation
- Penalty system for wrong submissions
- Contest timer logic

Frontend:

- Contest listing page
- Live contest interface
- Real-time leaderboard (polling every 30s)
- Contest countdown timer
- Post-contest results page

Deliverables:

- Students can view upcoming contests
- Live contests work with timer
- Leaderboard updates in real-time
- Final rankings displayed

Time Breakdown:

- Backend (2 days)
 - Frontend (2 days)
 - Testing with mock contest (1 day)
-

5.5 Phase 4: AI & Collaboration (Weeks 9-10)

Goal: AI helps students, teachers can monitor live

Week 9: AI Integration

Backend:

- Setup OpenAI API integration
- Implement rate limiting per user
- Create AI conversation storage

- Context-aware prompt engineering
- Caching layer for common queries

Frontend:

- Chatbot UI (sidebar or modal)
- Chat history display
- Code context sharing with AI
- "Explain this error" button in editor
- Rate limit indicator

Deliverables:

- ✅ AI chatbot responds to coding questions
- ✅ Can explain errors in student's code
- ✅ Rate limiting prevents abuse
- ✅ Chat history persists

Time Breakdown:

- OpenAI integration (1.5 days)
- Chat UI (2 days)
- Context awareness (1 day)
- Testing (0.5 day)

Week 10: Live Monitoring & Real-time Features

Backend:

- Setup Socket.io server
- Real-time submission broadcasting
- Online user presence tracking
- Live class view API

Frontend:

- Teacher live monitoring dashboard
- "Students online now" indicator
- Recent submissions feed (real-time)
- Broadcast message UI

Deliverables:

- ✅ Teachers see students online
- ✅ Submissions appear in real-time
- ✅ Can send announcements to class
- ✅ WebSocket connection stable

Time Breakdown:

- Socket.io setup (1 day)
- Live monitoring UI (2 days)
- Real-time feeds (1.5 days)
- Bug fixes (0.5 day)

5.6 Phase 5: Analytics & Polish (Weeks 11-12)

Goal: Beautiful analytics, progress tracking, and demo-ready polish

Week 11: Student Progress Intelligence (Demo Version)

Backend:

- Implement basic scoring algorithm
- Create StudySession, Goal models
- Calculate streak, time spent
- Weekly statistics API

Frontend:

- Progress dashboard with charts (Recharts)
- Goal setting interface
- Streak counter with fire emoji
- Weekly stats cards
- Simple Pomodoro timer

Deliverables:

- ✅ Students see overall progress score
- ✅ Can set goals manually
- ✅ Streak tracking works
- ✅ Study timer functional

Time Breakdown:

- Backend calculations (1.5 days)
 - Dashboard UI (2 days)
 - Timer implementation (1 day)
 - Polish (0.5 day)
-

Week 12: Teacher Analytics & Final Polish

Backend:

- Class analytics calculations
- Student performance aggregation
- Export CSV functionality
- Demo data seeding script

Frontend:

- Teacher analytics dashboard
- Class performance charts
- Top performers list
- Struggling students alerts
- Overall UI/UX polish
- Responsive design fixes
- Loading states everywhere
- Error handling improvements

Testing:

- Full user flow testing
- Cross-browser testing
- Mobile responsiveness
- Load testing with demo data

Documentation:

- User guide (screenshots + instructions)
- Teacher onboarding tutorial
- API documentation
- Demo video script

Deliverables:

- ✅ Beautiful teacher analytics
- ✅ Fully responsive on mobile
- ✅ No critical bugs
- ✅ Demo data populated
- ✅ User guide ready
- ✅ Demo video recorded

Time Breakdown:

- Analytics (2 days)
 - Polish & responsive (2 days)
 - Testing (0.5 day)
 - Documentation (0.5 day)
-

5.7 Post-Demo: Production Roadmap (Months 4-6)

Month 4: Advanced AI Features

- Code completion (Copilot-like)
- Hint system (Socratic mode)
- Test case generation
- Code review AI
- Fine-tuned model training

Month 5: Collaboration & Advanced Features

- Live collaborative editing (Google Docs for code)
- Video chat integration
- Screen sharing
- Advanced plagiarism detection (MOSS)
- White-label customization

Month 6: Scale & Enterprise

- Horizontal scaling setup
 - CDN for static assets
 - Database read replicas
 - Advanced monitoring (Grafana)
 - Payment integration (Stripe)
 - Enterprise SSO
 - Mobile apps (React Native)
-

6. Design System

6.1 Brand Identity

Platform Name: CodeLearn

Tagline: "Where Code Meets Intelligence"

Brand Personality:

- Modern & Professional
- Intelligent but Approachable
- Empowering & Motivating
- Clean & Minimalist

6.2 Color Palette

Primary Colors

```
--primary-900: #1e3a8a /* Deep Blue - Headers, CTAs */
--primary-700: #2563eb /* Blue - Primary actions */
--primary-500: #3b82f6 /* Bright Blue - Links, accents */
--primary-300: #93c5fd /* Light Blue - Hover states */
--primary-100: #dbeafe /* Very Light Blue - Backgrounds */
```

Semantic Colors

```
/* Success - Code accepted, progress */
--success-600: #16a34a /* Green */
--success-100: #dcfce7

/* Warning - Time running out, alerts */
--warning-600: #ca8a04 /* Amber */
--warning-100: #fef9c3

/* Error - Wrong answer, failures */
--error-600: #dc2626 /* Red */
--error-100: #fee2e2

/* Info - Tips, information */
--info-600: #0891b2 /* Cyan */
--info-100: #cffffe
```

Neutral Colors (Gray Scale)

```
--gray-950: #0f172a /* Almost black - Text */
--gray-800: #1e293b /* Dark gray - Secondary text */
--gray-600: #475569 /* Medium gray - Labels */
--gray-400: #94a3b8 /* Light gray - Placeholders */
--gray-200: #e2e8f0 /* Very light - Borders */
--gray-100: #f1f5f9 /* Background */
--gray-50: #f8fafc /* Card backgrounds */
```

Code Editor Theme (Dark Mode)

```
--editor-bg: #1e1e1e /* VS Code dark */
--editor-fg: #d4d4d4 /* Text */
--editor-selection: #264f78 /* Selection */
--editor-line-number: #858585
```

6.3 Typography

Font Families

```
/* Primary font - UI text */
--font-sans: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;

/* Monospace - Code, numbers */
--font-mono: 'JetBrains Mono', 'Fira Code', monospace;

/* Headings (optional accent) */
--font-heading: 'Poppins', 'Inter', sans-serif;
```

Font Sizes (Tailwind scale)

```
--text-xs: 0.75rem /* 12px - Captions */
--text-sm: 0.875rem /* 14px - Small text */
--text-base: 1rem /* 16px - Body */
--text-lg: 1.125rem /* 18px - Large body */
--text-xl: 1.25rem /* 20px - Headings */
--text-2xl: 1.5rem /* 24px - Page titles */
--text-3xl: 1.875rem /* 30px - Hero */
--text-4xl: 2.25rem /* 36px - Landing */
```

Font Weights

```
--font-normal: 400
--font-medium: 500
--font-semibold: 600
--font-bold: 700
```

6.4 Spacing System (Tailwind)

```
4px: 0.25rem (p-1, m-1)
8px: 0.5rem (p-2, m-2)
12px: 0.75rem (p-3, m-3)
16px: 1rem (p-4, m-4) ← Most common
24px: 1.5rem (p-6, m-6)
32px: 2rem (p-8, m-8)
48px: 3rem (p-12, m-12)
64px: 4rem (p-16, m-16)
```

Usage:

- **4px**: Icon spacing
- **8px**: Component padding
- **16px**: Card padding, button padding
- **24px**: Section spacing
- **48px**: Page section gaps

6.5 Component Specifications

Buttons

```

// Primary Button
<button className="
  px-4 py-2
  bg-primary-700 hover:bg-primary-800
  text-white font-medium
  rounded-lg
  transition-colors duration-200
  focus:outline-none focus:ring-2 focus:ring-primary-500 focus:ring-offset-2
">
  Submit Code
</button>

// Secondary Button
<button className="
  px-4 py-2
  bg-gray-200 hover:bg-gray-300
  text-gray-800 font-medium
  rounded-lg
  transition-colors duration-200
">
  Cancel
</button>

// Danger Button
<button className="
  px-4 py-2
  bg-error-600 hover:bg-error-700
  text-white font-medium
  rounded-lg
">
  Delete
</button>

// Icon Button
<button className="
  p-2
  rounded-lg
  hover:bg-gray-100
  transition-colors
">
  <Icon className="w-5 h-5" />
</button>

```

Cards

```

<div className="
  bg-white
  rounded-xl
  shadow-sm
  border border-gray-200
  p-6
  hover:shadow-md
  transition-shadow duration-200
">
  {/* Card content */}
</div>

```

Input Fields

```
<input className="
  w-full
  px-4 py-2
  border border-gray-300
  rounded-lg
  focus:outline-none focus:ring-2 focus:ring-primary-500 focus:border-transparent
  placeholder:text-gray-400
" />

// Error state
<input className="
  border-error-600
  focus:ring-error-500
" />
```

Badges

```
// Difficulty badges
<span className="px-2 py-1 text-xs font-semibold rounded-full bg-success-100 text-success-600">
  Easy
</span>

<span className="px-2 py-1 text-xs font-semibold rounded-full bg-warning-100 text-warning-600">
  Medium
</span>

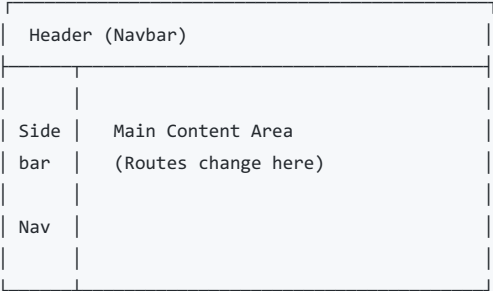
<span className="px-2 py-1 text-xs font-semibold rounded-full bg-error-100 text-error-600">
  Hard
</span>
```

Progress Bars

```
<div className="w-full bg-gray-200 rounded-full h-2">
  <div
    className="bg-primary-600 h-2 rounded-full transition-all duration-300"
    style={{ width: `${progress}%` }}
  />
</div>
```

6.6 Layout Structure

Student Dashboard Layout



Sidebar Width: 256px (w-64)
Header Height: 64px (h-16)
Main Content: Remaining space with max-width-7xl centered

Code Editor Layout

Problem Title Difficulty Stats	
Problem Description (Scrollable) Examples Constraints	Code Editor (Monaco)
	[Run] [Submit]
	Output Panel

Split: 40% Problem | 60% Editor+Output

6.7 Iconography

Icon Library: Lucide React

Common Icons:

```
import {
  Code,           // Coding/Editor
  BookOpen,       // Courses
  Trophy,         // Contests
  Target,         // Goals
  TrendingUp,     // Progress
  Users,          // Classes
  Settings,       // Settings
  Play,           // Run code
  Send,           // Submit
  Check,          // Success
  X,              // Error
  Clock,          // Time
  Flame,          // Streak
  Zap,            // Speed
  Brain,          // AI
  MessageSquare   // Chat
} from 'lucide-react';
```

Icon Sizes:

- Small: w-4 h-4 (16px) - Inline with text
- Medium: w-5 h-5 (20px) - Buttons
- Large: w-6 h-6 (24px) - Section headers
- XLarge: w-8 h-8 (32px) - Empty states

6.8 Animation & Transitions

Principles:

- Subtle and purposeful
- Duration: 150-300ms
- Easing: ease-in-out

```

/* Hover transitions */
transition: all 0.2s ease-in-out;

/* Page transitions */
transition: opacity 0.3s ease-in-out;

/* Skeleton loading */
@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}

```

Framer Motion Examples (Production):

```

// Page fade in
<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.3 }}
>

// Button press
<motion.button
  whileTap={{ scale: 0.95 }}
  whileHover={{ scale: 1.02 }}
>

// Modal
<motion.div
  initial={{ scale: 0.9, opacity: 0 }}
  animate={{ scale: 1, opacity: 1 }}
  exit={{ scale: 0.9, opacity: 0 }}
>

```

7. Data Models (Extended)

7.1 Relationships Diagram

```

User (Student/Teacher/Admin)
├──
├── Enrollments → Classes → Teacher (User)
├── CourseProgress → Courses → Lessons
├── Submissions → Problems → TestCases
├── Goals
├── StudySessions
├── AIConversations

Class
├── Enrollments (Students)
├── CourseAssignments → Courses
├── Teacher (User)

Contest
├── Problems (array of IDs)

```

7.2 Sample Data Seeding

For Demo (Week 12):

```
// seed.ts

const sampleCourses = [
  {
    title: "Python Fundamentals",
    difficulty: "BEGINNER",
    language: "Python",
    lessons: [
      { title: "Introduction to Python", content: "..."},
      { title: "Variables and Data Types", content: "..."},
      { title: "Control Flow", content: "..."},
      // ... 10 lessons total
    ]
  },
  {
    title: "Data Structures in Java",
    difficulty: "INTERMEDIATE",
    language: "Java",
    lessons: [...]
  },
  {
    title: "Advanced Algorithms",
    difficulty: "ADVANCED",
    language: "C++",
    lessons: [...]
  }
];

const sampleProblems = [
  {
    title: "Two Sum",
    difficulty: "EASY",
    topics: ["arrays", "hash-table"],
    description: "Given an array of integers...",
    testCases: [
      { input: "[2,7,11,15]\n9", output: "[0,1]", isSample: true },
      // ... 10 test cases total
    ]
  },
  // ... 50 problems total across difficulties
];

const sampleUsers = [
  {
    email: "teacher@demo.com",
    password: "demo123", // Hashed
    name: "Prof. Sarah Johnson",
    role: "TEACHER"
  },
  {
    email: "student@demo.com",
    password: "demo123",
    name: "Alex Chen",
    role: "STUDENT"
  },
  // ... 20 students total
];
```

8. Deployment Strategy

8.1 Demo Deployment (Free/Cheap Tier)

Frontend + API Routes:

```
Platform: Vercel
Plan: Hobby (Free)
Features:
- Automatic deployments from GitHub
- Edge functions
- Analytics
- Custom domain
Limits:
- 100 GB bandwidth/month
- 100 serverless function executions/day
- Good for 100-500 users
```

Database:

```
Platform: Neon (Serverless Postgres)
Plan: Free Tier
Features:
- 0.5 GB storage
- Auto-scaling
- Branching (dev/staging/prod)
Limits:
- 1 project
- 10 branches
- Good for demo
```

Redis:

```
Platform: Upstash
Plan: Free Tier
Features:
- 10,000 commands/day
- 256 MB storage
- REST API
Limits:
- Good for demo, need upgrade for production
```

File Storage:

```
Platform: Cloudflare R2
Plan: Free Tier
Features:
- 10 GB storage/month
- 1 million Class A operations
- S3-compatible API
Cost: $0 for demo
```

Code Execution:

```
Platform: Self-hosted Docker
Deployment: Railway (or Render)
Plan: $5/month
Features:
- 500 hours/month
- Persistent disk
- Custom Dockerfile
```

Total Demo Cost: ~\$5-10/month

8.2 Production Deployment

Frontend:

Platform: Vercel Pro
Cost: \$20/month
OR
Platform: AWS Amplify / Cloudflare Pages
Cost: ~\$15/month

Backend (API + Workers):

Platform: AWS EC2
Configuration:

- 2x t3.medium (4 vCPU, 4GB RAM)
- Load balanced (ALB)
- Auto-scaling group

Cost: ~\$60/month

Database:

Platform: AWS RDS (Postgres)
Configuration:

- db.t3.small (2 vCPU, 2GB RAM)
- Multi-AZ for redundancy
- Automated backups

Cost: ~\$30/month

Redis:

Platform: AWS ElastiCache
Configuration:

- cache.t3.micro

Cost: ~\$15/month

File Storage:

Platform: AWS S3 + CloudFront CDN
Storage: 100 GB
Transfer: 500 GB/month
Cost: ~\$20/month

Code Execution:

Option 1: Self-hosted Judge0
Platform: AWS EC2 t3.small
Cost: \$15/month

Option 2: Managed Judge0
Cost: \$0.0001/execution
Estimated: \$50/month (500k executions)

Monitoring:

Platform: Sentry (Error tracking)
Cost: \$26/month (Team plan)

Platform: Grafana Cloud (Metrics)
Cost: Free tier (or self-hosted)

Email:

Platform: Resend
Cost: \$20/month (50k emails)

AI (OpenAI):

Model: GPT-4o-mini

Usage:

- 100k students × 10 queries/day = 1M queries/month
- Avg 500 tokens/query

Cost: ~\$200/month

Optimization:

- Cache common Q&A (saves 50%)
- Rate limiting
- Actual cost: ~\$100/month

Total Production Cost: ~\$370/month (For 1000-5000 active users)

8.3 CI/CD Pipeline

```
# .github/workflows/deploy.yml

name: Deploy

on:
  push:
    branches: [main, staging]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
      - run: npm ci
      - run: npm run test
      - run: npm run lint

  deploy-staging:
    if: github.ref == 'refs/heads/staging'
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Vercel Staging
        run: vercel deploy --token=${{ secrets.VERCEL_TOKEN }}

  deploy-production:
    if: github.ref == 'refs/heads/main'
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Vercel Production
        run: vercel deploy --prod --token=${{ secrets.VERCEL_TOKEN }}
      - name: Run Database Migrations
        run: npm run prisma:migrate:deploy
```

8.4 Environment Configuration

```
# .env.example

# Database
DATABASE_URL="postgresql://user:pass@host:5432/codelearn"

# Authentication
NEXTAUTH_SECRET="your-secret-key-here"
NEXTAUTH_URL="https://codelearn.io"

# OAuth
GOOGLE_CLIENT_ID="..."
GOOGLE_CLIENT_SECRET="..."

# OpenAI
OPENAI_API_KEY="sk-..."

# Storage
R2_ACCOUNT_ID="..."
R2_ACCESS_KEY_ID="..."
R2_SECRET_ACCESS_KEY="..."
R2_BUCKET_NAME="codelearn-uploads"

# Redis
REDIS_URL="redis://default:password@host:6379"

# Code Execution
JUDGE0_URL="http://localhost:2358"
JUDGE0_API_KEY="..."

# Email
RESEND_API_KEY="re..."

# Monitoring
SENTRY_DSN="https://...@sentry.io/..."

# Feature Flags
ENABLE_AI_FEATURES="true"
ENABLE_CONTESTS="true"
```

9. Cost Analysis

9.1 Demo Phase Budget (Months 1-3)

Category	Service	Cost/Month	Notes
Hosting	Vercel (Hobby)	\$0	Free tier
Database	Neon	\$0	Free tier (3GB)
Redis	Upstash	\$0	Free tier
Storage	Cloudflare R2	\$0	10GB free
Code Execution	Railway	\$5	Docker containers
AI	OpenAI	\$50	Budget limit set
Domain	Namecheap	\$1	.com domain/year
Email	Resend	\$0	Free tier (3k/month)

Monitoring Category	Sentry Service	\$0 Cost/Month	Free tier Notes
Total		\$56/month	~\$170 for 3 months

User Capacity: 50-100 concurrent users

9.2 Production Phase Budget (Year 1)

Category	Service	Cost/Month	Annual
Hosting	Vercel Pro	\$20	\$240
Backend	AWS EC2 (2x)	\$60	\$720
Database	AWS RDS	\$30	\$360
Redis	ElastiCache	\$15	\$180
Storage	S3 + CloudFront	\$20	\$240
Code Execution	Judge0 Managed	\$50	\$600
AI	OpenAI	\$100	\$1,200
Email	Resend	\$20	\$240
Monitoring	Sentry	\$26	\$312
Domain & SSL		\$2	\$24
Backup	AWS S3	\$5	\$60
Total		\$348/month	\$4,176/year

User Capacity: 1,000-5,000 active users

9.3 Revenue Model & Break-Even Analysis

Pricing Tiers

Freemium (Individual Students):

Price: Free

Limits:

- 3 courses
- 50 problems/month
- 10 AI queries/day
- No contests

Goal: User acquisition, viral growth

Student Pro:

Price: \$10/month or \$80/year

Features:

- Unlimited courses
- Unlimited problems
- 50 AI queries/day
- All contests
- Certificates
- Priority support

Institution Basic:

Price: \$500/year (up to 100 students)

Features:

- All Pro features
- Teacher dashboard
- Class management
- Basic analytics
- Email support

Institution Pro:

Price: \$2,000/year (up to 500 students)

Features:

- All Basic features
- Advanced analytics
- White-label branding
- Custom domain
- Dedicated support
- API access

Enterprise:

Price: Custom (1000+ students)

Features:

- All Pro features
- SSO integration
- On-premise option
- SLA guarantee
- Training sessions

Break-Even Calculation

Monthly Operating Cost: \$348

Scenarios:

Option 1: Individual Students

- Need: 35 Student Pro subscriptions ($\$10 \times 35 = \350)
- Break-even: ~40 paying students

Option 2: Institutions

- Need: 1 Institution Pro (\$166/month)
- Or: 7 Institution Basic (\$42/month each)
- Break-even: 1 medium college

Option 3: Mixed (Realistic)

- 1 Institution Pro: \$166
- 2 Institution Basic: \$84
- 10 Student Pro: \$100

Total: \$350/month

Realistic Year 1 Target:

- 5 Institution Pro: \$833/month
- 10 Institution Basic: \$416/month
- 50 Student Pro: \$500/month
- **Total Revenue: \$1,749/month**
- **Profit: \$1,401/month (\$16,812/year)**

9.4 Funding Requirements

Bootstrap Approach (Recommended):

Phase 1 (Demo): \$170 (3 months)

- Self-funded or small grant
- Build MVP
- Get first 3 pilot customers (free)

Phase 2 (Launch): \$1,500 (6 months)

- Testimonials from pilots
- Charge first 10 customers
- Revenue covers costs by month 9

Phase 3 (Growth): Self-sustaining

- Reinvest profits
- Grow to 50 institutions
- \$25k+ MRR by end of year 1

External Funding (Optional):

Pre-Seed: \$50k

- 6-month runway
- Hire 1 developer
- Marketing budget
- Accelerated growth

Seed: \$200k (Year 2)

- Team of 5
- Aggressive marketing
- Enterprise features
- International expansion

10. Success Metrics (KPIs)

10.1 Product Metrics (Demo Phase)

Engagement:

- Daily Active Users (DAU)
- Weekly Active Users (WAU)
- Average session duration
- Pages per session

Learning:

- Course completion rate
- Average lessons completed per student
- Problems solved per student
- Contest participation rate

Retention:

- Day 1, Day 7, Day 30 retention
- Weekly returning users
- Churn rate

Technical:

- Page load time (< 2 seconds)
- API response time (< 500ms)
- Code execution time (< 3 seconds)
- Error rate (< 1%)

Demo Success Criteria:

- 50+ registered users
- 20+ DAU
- 3+ courses completed
- 100+ problems solved
- 90%+ uptime

10.2 Business Metrics (Production)

Growth:

- Monthly Recurring Revenue (MRR)
- MRR Growth Rate (target: 20%/month)
- Customer Acquisition Cost (CAC)
- Lifetime Value (LTV)
- LTV/CAC Ratio (target: > 3)

Conversion:

- Free to Paid conversion rate (target: 5%)
- Trial to Paid conversion (target: 25%)
- Activation rate (% who complete onboarding)

Customer Health:

- Net Promoter Score (NPS) (target: > 50)
- Customer Satisfaction Score (CSAT)
- Monthly churn rate (target: < 5%)
- Feature adoption rate

Year 1 Targets:

- 1,000 registered users
 - 100 paying customers
 - \$10,000 MRR
 - 95% customer satisfaction
-

10.3 Tracking Implementation

Tools:

Analytics: PostHog (self-hosted) or Mixpanel

- Event tracking
- Funnel analysis
- Cohort analysis

User Feedback: Typeform or Google Forms

- NPS surveys
- Feature requests
- Bug reports

Monitoring: Sentry + Grafana

- Error tracking
- Performance metrics
- Server health

A/B Testing: Growthbook (open-source)

- Feature flags
- Experiment tracking

Key Events to Track:

```
// User lifecycle
track('user_signed_up', { method: 'email' | 'google' })
track('user_completed_onboarding')
track('user_invited_classmate')

// Learning events
track('course_started', { courseId, difficulty })
track('lesson_completed', { lessonId, duration })
track('code_executed', { language, verdict })
track('problem_solved', { problemId, attempts, time })

// Engagement
track('ai_query_sent', { category: 'debug' | 'explain' | 'hint' })
track('contest_joined', { contestId })
track('code_shared', { snippetId })

// Business events
track('plan_upgraded', { from, to, price })
track('payment_succeeded', { amount, plan })
```

11. Risk Management

11.1 Technical Risks

Risk	Probability	Impact	Mitigation
Code execution security breach	Low	Critical	Sandboxed containers, regular security audits, bug bounty program
Database data loss	Low	Critical	Daily automated backups, point-in-time recovery, multi-AZ deployment
AI costs spiral out of control	Medium	High	Rate limiting, caching, budget alerts, tiered access
Platform downtime	Medium	High	Load balancing, auto-scaling, monitoring, 99.9% SLA
Scalability issues	Medium	Medium	Horizontal scaling architecture, performance testing, CDN

11.2 Business Risks

Risk	Probability	Impact	Mitigation
Low user adoption	Medium	Critical	Pilot programs, user research, iterate based on feedback
Competitors copy features	High	Medium	Focus on UX, community, move fast, patent core AI algorithms
Pricing too high/low	Medium	High	A/B test pricing, offer flexible tiers, annual discounts
Institutions slow to adopt	High	High	Free trials, case studies, ROI calculators, reference customers
Churn from bugs	Medium	High	Rigorous testing, fast bug fixes, proactive support

11.3 Compliance & Legal

Data Privacy:

- GDPR compliance (for European users)
- COPPA compliance (for users under 13)
- Data encryption at rest and in transit

- Clear privacy policy and terms of service

Intellectual Property:

- Trademark "CodeLearn" name and logo
- Copyright all course content
- Open-source components properly licensed

Security:

- Regular penetration testing
- Vulnerability disclosure program
- Compliance with OWASP Top 10

12. Innovation Features (My Additions)

12.1 Learning Path Visualizer

Concept: Interactive skill tree showing course dependencies

Implementation (Production):

```
// Uses D3.js force-directed graph
const SkillTree = () => {
  const nodes = [
    { id: 'python-basics', level: 1, unlocked: true },
    { id: 'data-structures', level: 2, unlocked: true },
    { id: 'algorithms', level: 3, unlocked: false },
    // ...
  ];

  const links = [
    { source: 'python-basics', target: 'data-structures' },
    { source: 'data-structures', target: 'algorithms' },
  ];

  // Render interactive graph
  // Click node → Show course details
  // Grayed out = locked (need prerequisites)
};
```

Value Prop: Students see clear path to their goal, gamified progression

12.2 Code Journey Replay

Concept: Timelapse of student's coding process

Implementation:

```
// Store code snapshots every 30 seconds
interface CodeSnapshot {
  timestamp: Date;
  code: string;
  cursorPosition: number;
  event: 'typed' | 'deleted' | 'ran' | 'error';
}

// Replay with controls (play, pause, speed)
const CodeReplay = ({ snapshots }) => {
  const [currentIndex, setCurrentIndex] = useState(0);

  // Show diff highlighting between snapshots
  // AI analysis: "Spent 40% time on line 23 logic error"
};
```

Value Prop: Teachers understand student thought process, students learn from mistakes

12.3 Socratic Tutor Mode

Concept: AI asks guiding questions instead of giving answers

Prompt Engineering:

```
const socraticPrompt = `
You are a Socratic coding tutor. When a student asks for help:
1. NEVER give direct answers or complete code
2. Ask probing questions that guide their thinking
3. Break down the problem into smaller steps
4. If they're stuck, give tiny hints through questions
5. Celebrate when they figure it out themselves

Example:
Student: "Why is my loop not working?"
You: "What do you expect i to be when the loop ends? What is it actually?"
Student: "It should be 10 but it's 11"
You: "Interesting! What condition are you checking in your loop? When does it stop?"
`;

const chatWithAI = async (message: string) => {
  const response = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    messages: [
      { role: 'system', content: socraticPrompt },
      { role: 'user', content: message }
    ]
  });
  return response.choices[0].message.content;
};
```

Value Prop: Builds critical thinking, prevents AI dependency

12.4 Problem Generator AI

Concept: Infinite unique problems based on student's level

Implementation:

```

const generateProblem = async (criteria: {
  topic: string;
  difficulty: 'easy' | 'medium' | 'hard';
  studentLevel: number;
}) => {
  const prompt = `
    Generate a unique ${criteria.difficulty} coding problem about ${criteria.topic}.
    Student level: ${criteria.studentLevel}/100

    Return JSON:
    {
      "title": "...",
      "description": "...",
      "examples": [...],
      "constraints": [...],
      "testCases": [
        { "input": "...", "output": "...", "explanation": "..." }
      ],
      "hints": ["...", "..."]
    }
  `;

  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [{ role: 'user', content: prompt }],
    response_format: { type: 'json_object' }
  });

  return JSON.parse(response.choices[0].message.content);
};

```

Value Prop: Personalized practice, never run out of problems

12.5 Skills Constellation Visualizer

Concept: 3D interactive graph of student's skill mastery

Implementation (Three.js):

```

const SkillsConstellation = ({ skills }) => {
  // Each skill is a planet
  // Size = mastery level (0-100%)
  // Glow intensity = recent activity
  // Orbit speed = practice frequency
  // Connections = related skills

  const planets = skills.map(skill => ({
    position: calculatePosition(skill.category),
    size: skill.mastery / 10,
    color: getColorByMastery(skill.mastery),
    rotationSpeed: skill.recentActivity / 100
  }));

  return (
    <Canvas>
      <StarField />
      {planets.map(planet => (
        <Planet {...planet} onClick={() => showSkillDetails(planet)} />
      ))}
      <OrbitControls />
    </Canvas>
  );
};

```

Value Prop: Beautiful, engaging progress visualization

12.6 Predictive Intervention System

Concept: AI predicts which students will struggle before they fail

Machine Learning Model:

```
# Features
features = [
    'avg_session_duration',
    'problems_solved_per_week',
    'streak_length',
    'submission_accuracy',
    'time_between_sessions',
    'quiz_scores',
    'help_requests_count',
    'contest_participation'
]

# Train on historical data
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(historical_data[features], historical_data['passed_course'])

# Predict risk score for current students
risk_scores = model.predict_proba(current_students[features])

# Alert teacher if risk > 80%
at_risk_students = students[risk_scores[:, 1] > 0.8]
send_alert_to_teacher(at_risk_students, suggested_interventions)
```

Value Prop: Proactive teaching, prevent failures before they happen

13. Go-to-Market Strategy

13.1 Pre-Launch (Weeks 0-12)

Objectives:

- Build MVP
- Validate with real users
- Generate testimonials

Tactics:

1. Pilot Program

- Approach 3-5 local colleges
- Offer free access for 1 semester
- In exchange: Feedback + testimonials

2. Build in Public

- Tweet progress daily (#buildingpublic)
- Write blog posts on dev journey
- Share on Reddit (r/learnprogramming, r/coding)

3. Content Creation

- "How I built a coding platform" YouTube series
 - Technical blog: "Scaling code execution"
 - Guest posts on dev.to, Hashnode
-

13.2 Launch (Months 4-6)

Objectives:

- 1,000 registered users
- 50 paying customers
- \$2,000 MRR

Tactics:

1. **Product Hunt Launch**

- Prepare: Teaser 2 weeks before
- Launch day: Engage in comments
- Offer: 50% off for first 100 signups

2. **Freemium Funnel**

- Free tier: Attracts students
- Upsell to Pro: After they solve 50 problems
- Institutional sales: Reach out to active students' colleges

3. **Influencer Marketing**

- Partner with coding YouTubers (reviews)
- Sponsor CS student communities
- Offer affiliate program (20% commission)

4. **SEO Content**

- "Top 100 coding interview questions" (with our problems)
- "How to learn Python in 2026"
- "Best coding practice platforms compared"

13.3 Growth (Months 7-12)

Objectives:

- 10,000 registered users
- 100 paying institutions
- \$10,000 MRR

Tactics:

1. **Direct Sales**

- Hire 1 sales person
- Cold email to CS department heads
- Attend education conferences
- Webinars: "How to improve placement rates"

2. **Referral Program**

- Give 2 months free for each referral
- Teacher referral bonus: \$100 per institution

3. **Case Studies**

- "How XYZ College increased placements by 40%"
- Video testimonials from teachers
- Student success stories

4. **Partnerships**

- Integrate with Learning Management Systems (Moodle, Canvas)
- Partner with coding bootcamps
- Corporate training programs

14. Team & Roles (Future Hiring)

Current (Demo Phase)

- **You:** Full-stack developer, Product Manager, Designer

After Launch (Month 4+)

- **Frontend Developer:** React/Next.js specialist
- **Backend Developer:** API, database optimization
- **Designer:** UI/UX, brand identity
- **Content Creator:** Course content, problems

Growth Phase (Month 9+)

- **Sales Manager:** B2B institutional sales
- **Customer Success:** Onboarding, support

- **DevOps Engineer:** Infrastructure, scaling
 - **Data Scientist:** ML models, analytics
-

15. Next Steps (Immediate Action Plan)

This Week (Days 1-7)

Day 1-2: Validation

- ☐ Interview 5 students about current learning pain points
- ☐ Interview 3 teachers about grading/teaching challenges
- ☐ Validate pricing: Would they pay \$X?

Day 3-4: Setup

- ☐ Create GitHub repository
- ☐ Setup Linear/Notion for project management
- ☐ Create Figma account for designs
- ☐ Buy domain name

Day 5-7: Design

- ☐ Sketch wireframes (paper/Figma)
 - ☐ Design login/register pages
 - ☐ Design student dashboard mockup
 - ☐ Design course page mockup
-

Week 2-3: Development Begins

- ☐ Initialize Next.js project
 - ☐ Setup Prisma + database
 - ☐ Implement authentication
 - ☐ Build basic UI components
 - ☐ Deploy to Vercel staging
-

16. Conclusion

Why This Will Succeed

1. Real Problem:

- Students struggle to learn coding effectively
- Teachers overwhelmed with manual grading
- Institutions need measurable outcomes

2. Unique Solution:

- AI-first approach (not bolted on later)
- Holistic (learn + practice + compete)
- Institutional focus (B2B + B2C)

3. Market Timing:

- AI boom makes this possible now
- Remote learning normalized post-COVID
- Demand for tech talent at all-time high

4. Execution Plan:

- Start small (demo in 12 weeks)
 - Validate with real users early
 - Iterate based on feedback
 - Scale with revenue
-

Your Competitive Advantages

1. **Speed:** Build and iterate faster than competitors
2. **Focus:** Niche down to coding education vs general LMS
3. **Innovation:** AI features competitors will take years to build
4. **Pricing:** Affordable for emerging markets (India, SEA)

Final Motivation

"The best time to start was yesterday. The second best time is now."

You have:

- ☑ A validated idea (inspired by PlayCode + your insights)
- ☑ Clear roadmap (12 weeks to demo)
- ☑ Affordable budget (\$170 for demo)
- ☑ Differentiated features (AI, progress tracking)
- ☑ Realistic path to revenue (\$10k MRR Year 1)

What's needed: Execution

Start with Week 0. Interview users. Build conviction. Then code relentlessly for 12 weeks.

By March 2026, you'll have a demo that makes people say: **"This is exactly what we needed!"**

Appendix A: Glossary

- **AC:** Accepted (correct solution)
 - **API:** Application Programming Interface
 - **CRUD:** Create, Read, Update, Delete
 - **CTR:** Click-through Rate
 - **DAU:** Daily Active Users
 - **DP:** Dynamic Programming
 - **LMS:** Learning Management System
 - **LSP:** Language Server Protocol
 - **MRR:** Monthly Recurring Revenue
 - **NPS:** Net Promoter Score
 - **ORM:** Object-Relational Mapping
 - **RBAC:** Role-Based Access Control
 - **RE:** Runtime Error
 - **SLA:** Service Level Agreement
 - **SSO:** Single Sign-On
 - **TLE:** Time Limit Exceeded
 - **WA:** Wrong Answer
 - **WAU:** Weekly Active Users
-

Appendix B: Resources

Learning Resources

- Next.js: <https://nextjs.org/learn>
- Prisma: <https://www.prisma.io/docs>
- Monaco Editor: <https://microsoft.github.io/monaco-editor/>
- Judge0: <https://judge0.com/>
- OpenAI API: <https://platform.openai.com/docs>

Inspiration

- LeetCode: <https://leetcode.com>
- Replit: <https://replit.com>
- CodeHS: <https://codehs.com>
- HackerRank: <https://hackerrank.com>

Design

- shadcn/ui: <https://ui.shadcn.com>
- Tailwind: <https://tailwindcss.com>
- Lucide Icons: <https://lucide.dev>

Community

- r/learnprogramming
 - r/coding
 - Indie Hackers
 - Product Hunt
-

Document End

Ready to build something amazing? Let's go! 🚀